

Ain Shams University
Faculty of Engineering
Computer Engineering and Software Systems



CFI - Care: Collaborative Flow of Intelligence for Care

Graduation Project Thesis

Author: Mohamed Salah Fathy 21P0117
Youssef Tamer Hossam 21P0138
Salma Hisham Hassan Wagdy 21P0124
Salma Mohamed Abdelmonem 21P0148
Belal Anas Awad 21P0072
Supervisors: Dr Haitham Azmi

Submission Date: 04 April, 2025

Mohamed Salah Fathy 21P0117
Youssef Tamer Hossam 21P0138
Salma Hisham Hassan Wagdy 21P0124
Salma Mohamed Abdelmonem 21P0148
Belal Anas Awad 21P0072
Date: 04 April, 2025

Abstract

Healthcare today is characterized by fragmented experiences where patient medical information is often dispersed across various institutions and providers. This project proposes a comprehensive, patient-centric healthcare application designed to consolidate all medical reports into a single, secure, and unified platform. The system employs Role-Based Access Control (RBAC) to ensure privacy and controlled information sharing. To enhance clinical decision-making, the platform features a medical history visualization graph for tracking health trends and integrates a Retrieval-Augmented Generation (RAG)-enabled AI model to generate diagnostic reports and insights. By transforming isolated data into an interconnected ecosystem, this project aims to establish meaningful relationships between patients, caregivers, and doctors for personalized healthcare delivery.

Contents

1	Introduction	1
1.1	Digital Health Market Overview	3
1.2	HL7 - Health Level Seven	3
1.2.1	Evolution of Standards	3
1.3	Fast Healthcare Interoperability Resources (FHIR)	4
1.3.1	Core Architecture	4
1.3.2	Key Technical Advantages	4
1.3.3	Comparison of Standards	5
2	Literature Review	7
3	Proposed Design Architecture	8
3.1	System Architecture	8
3.1.1	High-Level Architecture	8
3.1.2	Node.js Application Server	9
3.1.3	Data Persistence and Schema	10
3.1.4	Security and Infrastructure Layer	12
3.1.5	HAPI FHIR Server Implementation	12
3.2	Patient History Graph Implementation	13
3.2.1	Graph Construction Algorithm	13
3.2.2	Temporal Branching	14
3.3	Performance Optimization	14
3.3.1	Tier 1: Standard Resource Cache	14
3.3.2	Tier 2: Specialized Graph Cache	14
3.4	Frontend	14
3.5	Mobile	14
4	Security Framework	15
5	AI Integration	16
6	Testing and Verification	17
7	Conclusion and Future Work	18

Appendix	19
A Lists	20
List of Abbreviations	20
List of Figures	21
References	22

Chapter 1

Introduction

Today, patients often face fragmented healthcare experiences, with their medical information dispersed across multiple institutions and providers. This project proposes a patient-centric healthcare application designed to consolidate all medical reports within a single, secure, and unified platform.

The system empowers patients to manage their health through continuous data collection, secure communication, and controlled information sharing using Role-Based Access Control (RBAC) to ensure privacy, consent, and data integrity. Additionally, by integrating features such as medical history visualization and Retrieval-Augmented Generation (RAG)-enabled AI, the proposed system transforms previously scattered and isolated medical data into a unified, interconnected ecosystem. This integration establishes meaningful relationships between patients, healthcare providers, and medical records, promoting continuous, coordinated, and personalized healthcare delivery.

Purpose

The primary purpose of this project is to develop a unified healthcare platform that securely consolidates patients' medical records from multiple institutions. By leveraging technologies such as RAG-enabled AI and graphical data visualization, the project aims to enhance clinical insight and support medical decision-making. Furthermore, the platform seeks to empower patients to monitor their own health data while maintaining full control over information sharing through robust RBAC mechanisms. Ultimately, this initiative strives to ensure privacy, data integrity, and interoperability across the healthcare ecosystem to enhance patient trust and system reliability.

Objective

The key objectives of this project are as follows:

- To develop a unified healthcare platform that securely consolidates patients' medical records from multiple institutions.
- To empower patients to manage and monitor their own health data while maintaining full control over information sharing through RBAC mechanisms.
- To enable secure and efficient communication between patients, doctors, and caregivers.
- To provide data visualization tools, including a medical history graph, to help healthcare providers interpret patient data trends and clinical progress.
- To integrate RAG-enabled AI capabilities to generate automated reports and actionable insights that support doctors in diagnosis and medical decision-making.
- To ensure privacy, data integrity, and interoperability across the healthcare ecosystem to enhance patient trust and system reliability.

1.1 Digital Health Market Overview

The global push towards digital transformation in healthcare is driven by the need for efficiency, accessibility, and improved patient outcomes. According to Grand View Research, the global digital health market size was valued at USD 288.55 billion in 2024 and is projected to reach USD 946.04 billion by 2030, expanding at a Compound Annual Growth Rate (CAGR) of 22.2% from 2025 to 2030 [1].

This rapid growth is fueled by factors such as the increasing penetration of smartphones, advancements in internet connectivity, and government initiatives promoting digital health infrastructure [1]. However, realizing this potential requires overcoming the challenge of data silos. Currently, healthcare data is often locked within proprietary systems, making *interoperability*—the ability of different information systems to access, exchange, integrate, and cooperatively use data in a coordinated manner—a critical requirement for the modern healthcare ecosystem [3].

1.2 HL7 - Health Level Seven

Health Level Seven (HL7) International is a standards development organization that has defined the framework for the exchange, integration, sharing, and retrieval of electronic health information for decades [2]. Understanding the evolution of these standards is crucial to appreciating the architectural shifts in modern healthcare applications.

1.2.1 Evolution of Standards

- **HL7 v2 (The Workhorse):** Established in 1989, HL7 v2 remains the most widely implemented standard for hospital workflows (e.g., ADT - Admission, Discharge, Transfer messages). It utilizes a non-XML, pipe-delimited syntax (e.g., MSH|^~\&|...). While highly efficient for legacy backend systems, its lack of semantic strictness leads to significant implementation variances, posing challenges for modern web and mobile application development [2].
- **HL7 v3 (The Theoretical Model):** Introduced in the 1990s to address the vagueness of v2, HL7 v3 was built on a rigorous Reference Information Model (RIM). It enforced strict semantic correctness using complex XML structures. However, this theoretical purity resulted in a steep learning curve and bloated message sizes, leading to poor adoption rates and a lack of backward compatibility with v2 [2].
- **HL7 CDA (Clinical Document Architecture):** A component of v3, CDA focuses on the exchange of clinical documents (such as discharge summaries). While effective for static snapshots of care, it is document-centric rather than data-centric, making it difficult to extract granular data (like a single heart rate reading) required for real-time interactive applications [4].

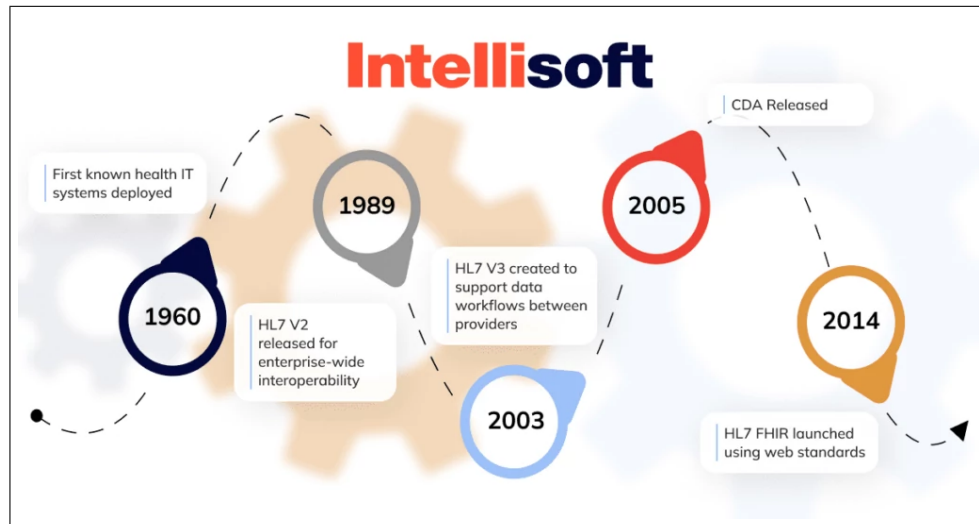


Figure 1.1: Evolution of HL7 Standards from Messaging to RESTful Resources.

1.3 Fast Healthcare Interoperability Resources (FHIR)

In response to the complexity of HL7 v3 and the limitations of v2, HL7 introduced **FHIR** (Fast Healthcare Interoperability Resources) in 2012. FHIR represents a paradigm shift from complex, custom healthcare protocols to modern web standards.

1.3.1 Core Architecture

Unlike its predecessors, FHIR is built on the RESTful API approach widely used in modern IT industries (e.g., by Google, Facebook, Twitter). It treats data elements as independent "Resources"—modular components like **Patient**, **Observation**, or **Medication**—that can be accessed and manipulated via standard HTTP methods (GET, POST, PUT, DELETE) [4].

1.3.2 Key Technical Advantages

- **Granularity via Resources:** FHIR allows systems to access specific data elements using unique URLs (e.g., `GET /Patient/123`), rather than exchanging entire heavy documents. This makes it ideal for mobile devices with limited bandwidth [4].
- **Modern Formats (JSON & XML):** While previous standards relied on complex XML or pipe-delimited text, FHIR natively supports JSON (JavaScript Object Notation), which is lightweight and directly parsable by web browsers and mobile apps [2].

- **The 80/20 Rule:** FHIR focuses on defining the 20% of requirements that satisfy 80% of common healthcare use cases. For the remaining complex or local requirements, it uses a built-in extension mechanism, ensuring the core standard remains simple and implementable [3].

1.3.3 Comparison of Standards

Table 1.1 highlights the fundamental differences between the major HL7 standards, illustrating why FHIR was chosen as the interoperability backbone for this project.

Table 1.1: Comparison of HL7 v2, v3, CDA, and FHIR Standards

Feature	HL7 v2	HL7 v3 / CDA	FHIR
Primary Paradigm	Event-Driven Messaging	Reference Model (RIM) / Documents	RESTful API & Resources
Data Format	Pipe-delimited text	Complex XML	JSON, XML, RDF
Transport	LLP / MLLP (TCP/IP)	SOAP / Web Services	HTTP / REST
Complexity	Moderate (Vague semantics)	High (Steep learning curve)	Low (Web-standard based)
Use Case	Hospital Backend (ADT, Orders)	Clinical Documents (Summaries)	Mobile, Web, Cloud, & Backend

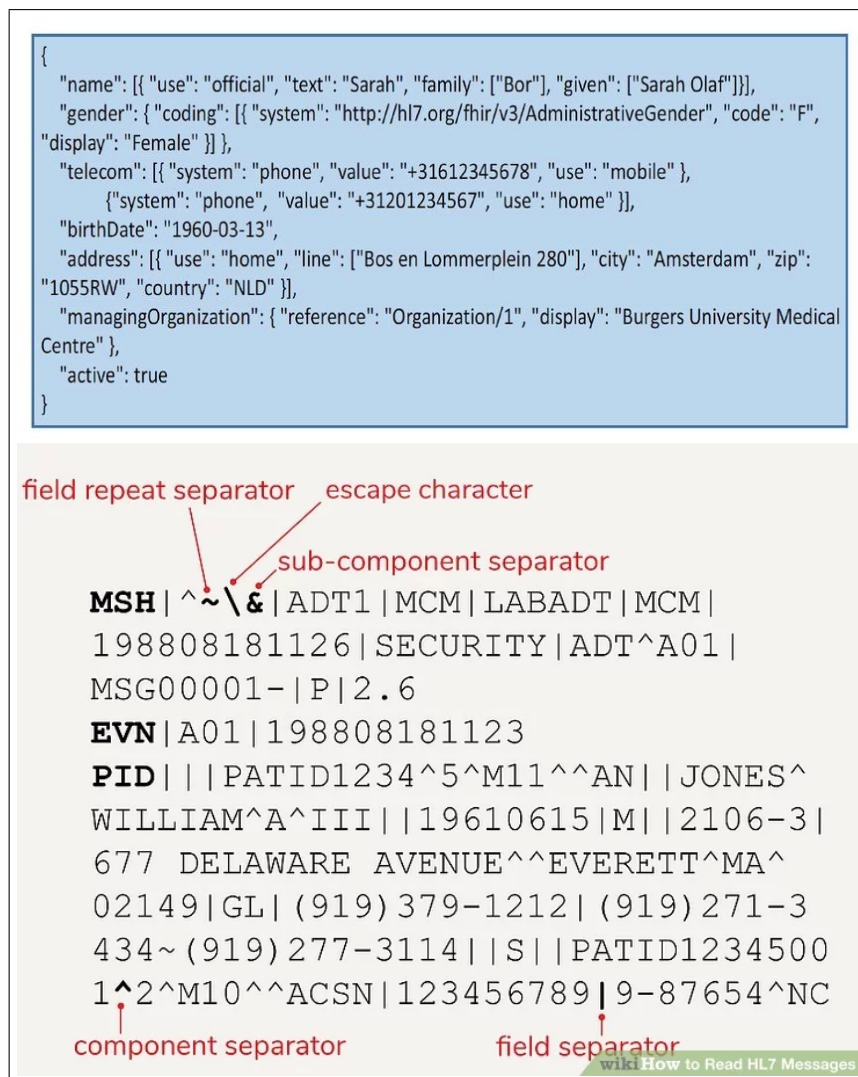


Figure 1.2: Visual Comparison of Data structure: HL7 v2 Message vs. FHIR JSON Resource.

Chapter 2

Literature Review

Chapter 3

Proposed Design Architecture

3.1 System Architecture

The CFI-Care system is built upon a robust three-tier architecture designed to facilitate secure and interoperable healthcare data management. The system integrates a modern frontend, a logic-heavy middleware layer, and a standards-compliant backend to ensure data consistency and security. The architecture adheres to the FHIR (Fast Healthcare Interoperability Resources) standard for all medical data exchange and utilizes OAuth 2.0 via Keycloak for identity management.

3.1.1 High-Level Architecture

The application operates through a strict separation of concerns. The Presentation Layer is an Angular-based Single Page Application (SPA) that serves as the user interface. It does not communicate directly with the data stores. Instead, all client requests are intercepted by an Nginx Reverse Proxy, which handles SSL/TLS termination and routes traffic to the appropriate internal services.

The core of the backend processing is handled by the Node.js Application Server. This server acts as the primary API gateway, implementing business logic, enforcing security policies, and managing user sessions. It communicates with the HAPI FHIR Server, a specialized Java-based component responsible for persisting clinical data in compliance with the FHIR R5 standard. Data storage is consolidated in PostgreSQL databases, while high-speed data access and session management are handled by Redis instances.

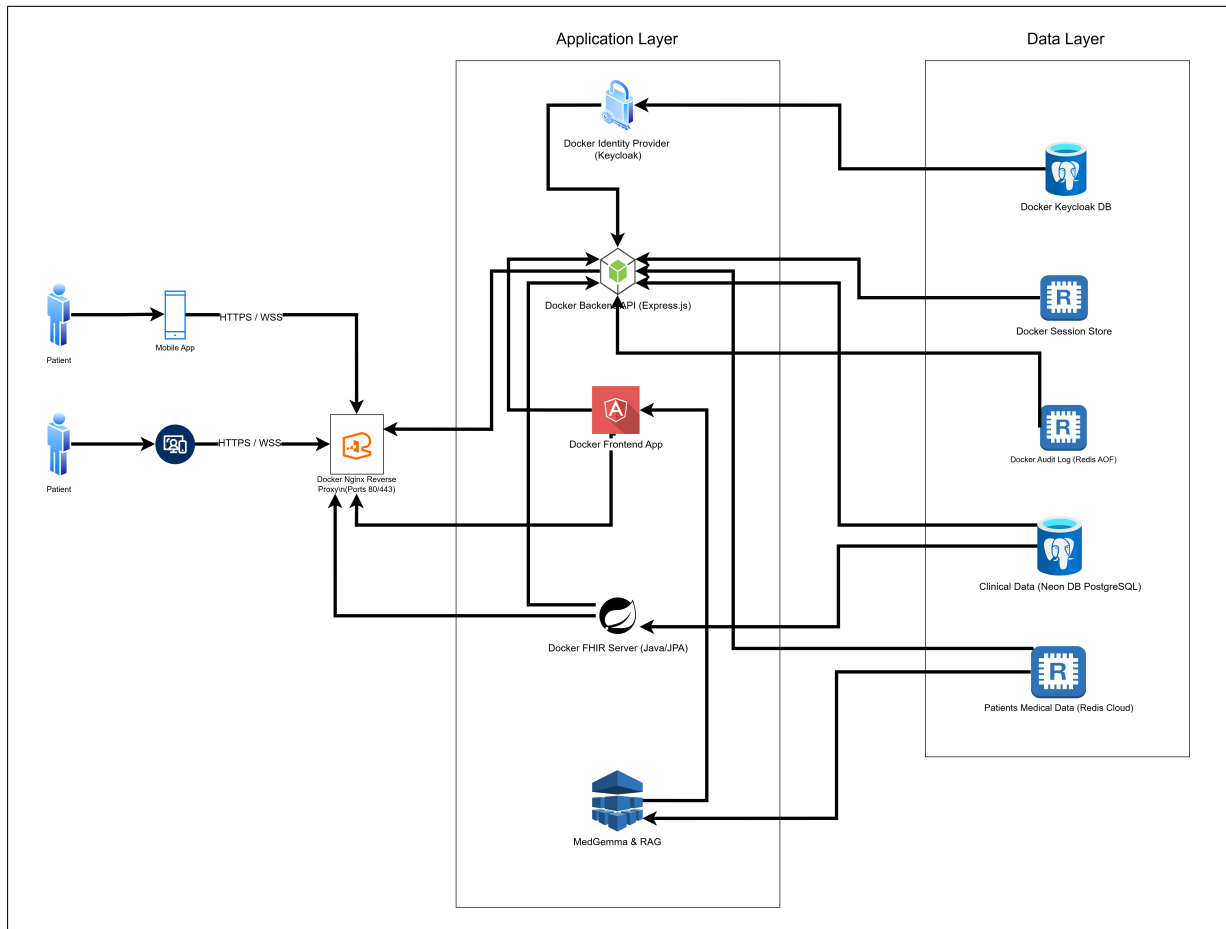


Figure 3.1: High-Level Container Architecture showing the interaction between the Nginx proxy, application services, and data persistence layers.

3.1.2 Node.js Application Server

The Node.js server, built with the Express.js framework, serves as the middleware layer that bridges the gap between the user interface and the standardized medical records. It is designed with a modular layered architecture to ensure scalability and maintainability.

Middleware and Security

Upon receiving a request, the server passes it through a comprehensive middleware stack. Security headers are applied using Helmet to prevent common web vulnerabilities. Cross-Origin Resource Sharing (CORS) is configured to restrict access to authorized domains. Crucially, the server implements rate limiting to protect API endpoints from abuse and utilizes a Redis-backed session store to manage user authentication states efficiently.

Routing and Services

The application is divided into specific route modules corresponding to healthcare domains. Dedicated endpoints manage Patients, Practitioners, Encounters, and Episodes of Care. A specialized "History Graph" module handles the retrieval and processing of graph visualization data.

Beneath the routing layer lies the service layer, which contains the core business logic. These services act as proxies to the FHIR server, transforming raw FHIR JSON into formats optimized for the frontend, such as the D3.js visualizations. They also handle document processing tasks, such as converting base64 strings to PDF files for binary storage.

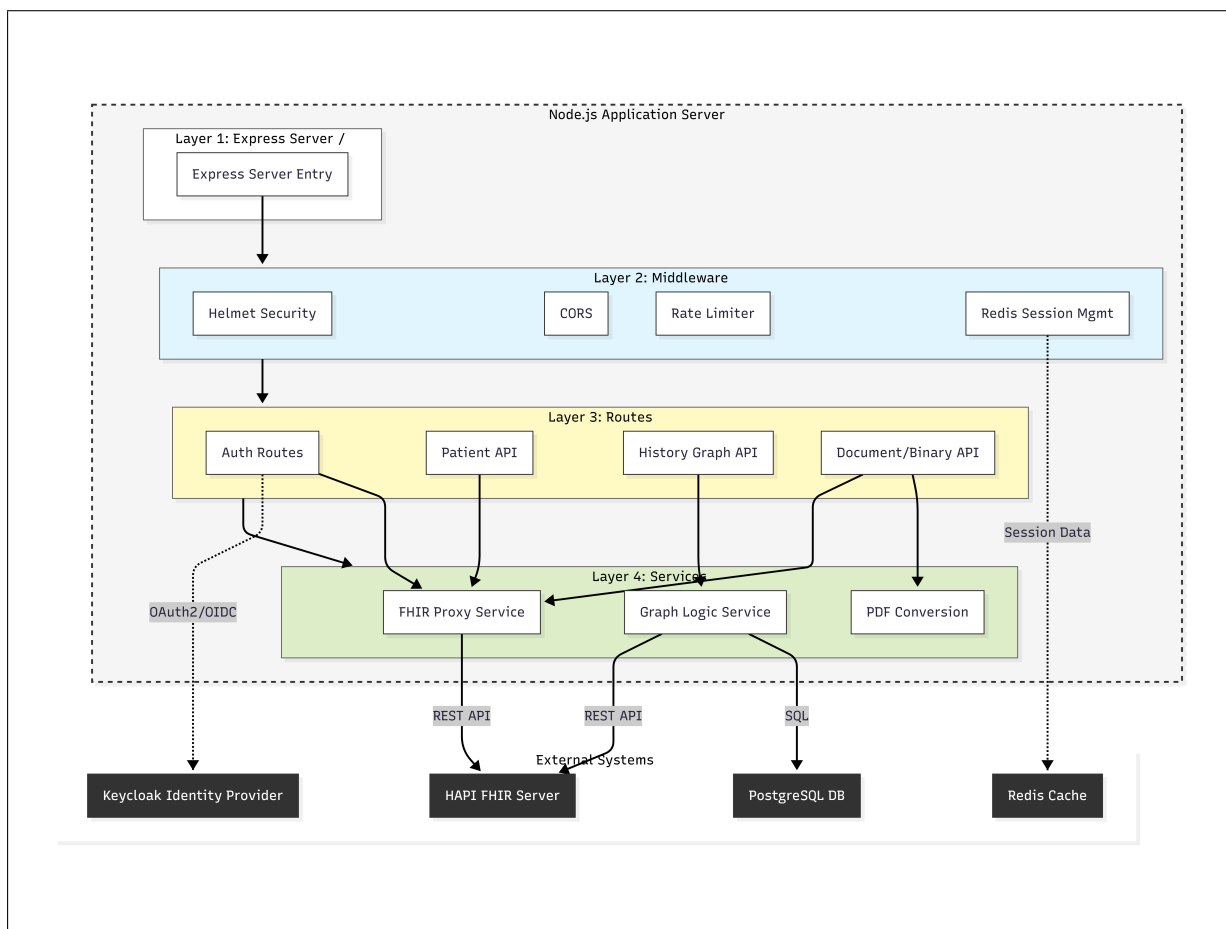


Figure 3.2: Internal layered architecture of the Node.js Application Server, detailing the middleware stack and service distribution.

3.1.3 Data Persistence and Schema

The system utilizes a polyglot persistence strategy, employing different database technologies for specific use cases to maximize performance.

Relational Data

The primary storage engine is PostgreSQL. It is divided into two logical segments. The first segment supports the HAPI FHIR server, storing all standard clinical resources (like Patients and Observations) in a highly structured format managed by Hibernate. The second segment supports the Keycloak identity provider, storing user credentials and realm configurations.

Custom Graph Schema

To support the unique "Medical History Graph" feature, a custom schema was designed alongside the standard FHIR tables. This schema consists of **encounter_nodes** and **node_relations**. These tables allow the system to map standard FHIR Encounters into a Directed Acyclic Graph (DAG), enabling the visualization of a patient's journey through time with causal links between medical events.

Caching and Auditing

Redis is utilized for ephemeral data storage. One instance is dedicated to storing user sessions, ensuring fast authentication checks. A second Redis instance operates in Append-Only File (AOF) mode to store audit logs, providing a persistent and high-speed trail of system activities.

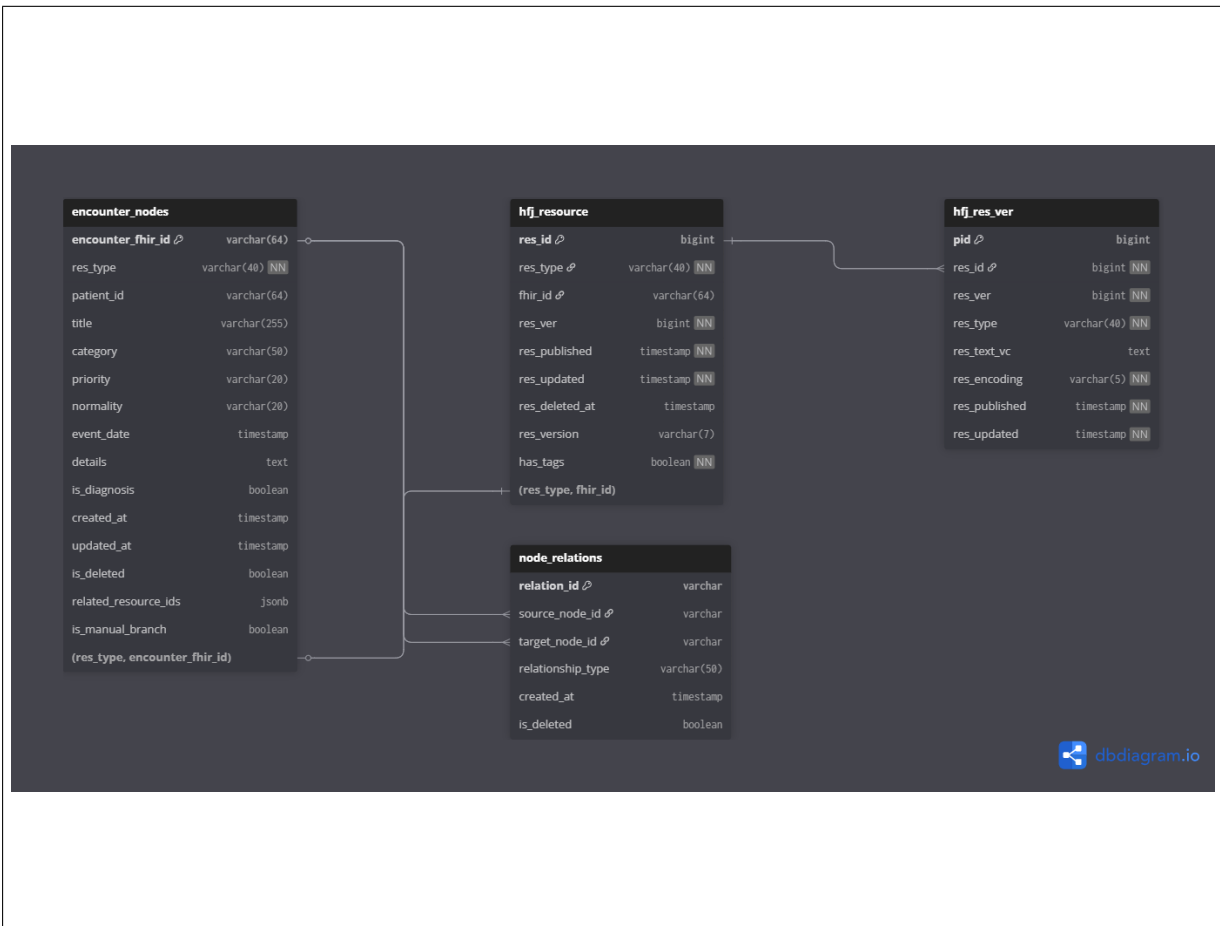


Figure 3.3: Entity-Relationship Diagram (ERD) illustrating the integration of custom graph tables with the standard FHIR resource table.

3.1.4 Security and Infrastructure Layer

The infrastructure is secured through a centralized Identity and Access Management (IAM) system using Keycloak. Keycloak handles all OAuth 2.0 and OpenID Connect flows, issuing JSON Web Tokens (JWT) that the Node.js backend validates against a public key set (JWKS).

All components are containerized using Docker and orchestrated via Docker Compose. The environment is isolated within a private bridge network, with Nginx acting as the sole entry point. Nginx enforces SSL/TLS encryption using locally generated certificates, ensuring that all data in transit is encrypted. Containers are configured with read-only file system mounts and strict privilege limitations to minimize the security attack surface.

3.1.5 HAPI FHIR Server Implementation

The core component responsible for standardizing healthcare data within the CFI-Care system is the FHIR Server. This server is implemented using the HAPI FHIR JPA Server

Starter framework. It is built upon the Java Spring Boot ecosystem and provides a fully compliant implementation of the FHIR Release 5 (R5) standard. This choice ensures that all clinical data, regardless of its source, adheres to international interoperability protocols and persists reliably within the database.

To tailor the server for the specific needs of this project, several important configuration changes were applied to the default setup. To maintain a lightweight and efficient runtime, advanced features that were not immediately required were explicitly disabled. This includes the Clinical Reasoning module and the CDS Hooks service. Similarly, bulk export and import operations were deactivated to prevent unnecessary resource consumption. Conversely, the OpenAPI documentation endpoints were enabled. This configuration allows developers to inspect the API structure and test endpoints directly through a web interface.

The persistence layer is powered by a PostgreSQL database connected via the Hibernate JPA engine. Specific configurations were applied to the Hibernate settings to optimize performance. The system uses the dedicated `HapiFhirPostgresDialect` to ensure efficient SQL generation. Additionally, the database schema management is configured to automatically update the table structures upon server startup, ensuring that the database remains synchronized with the code.

The server is responsible for managing a wide array of clinical resources necessary for the application. These include foundational entities such as Patient and Practitioner, as well as clinical events like Encounters, Procedures, and Observations. It also handles complex data types, including MedicationRequests for prescriptions and Binary resources for storing documents like PDF files.

For deployment, the server utilizes a secure multi-stage Docker build process. The first stage uses Maven to compile the application code and generate the necessary artifacts. The final runtime image is built upon a "distroless" Java 17 base image. This approach significantly reduces the final container size and improves security by removing unnecessary operating system tools and shells from the production environment.

3.2 Patient History Graph Implementation

A core distinguishing feature of the CFI-Care backend is the Patient History Graph, which visualizes a patient's medical journey as a Directed Acyclic Graph (DAG). Unlike standard FHIR lists, which present data linearly, this graph models causal and temporal relationships between medical events.

3.2.1 Graph Construction Algorithm

The graph logic is encapsulated within the `historyGraphService`. When a new medical event (Node) is recorded, the system performs a temporal scan of existing nodes for that patient. It identifies potential "Parent" nodes based on the event date and clinical category. For example, a "Prescription" node is automatically linked to the most recent "Consultation" node, creating a parent-child relationship in the `node_relations` table.

3.2.2 Temporal Branching

To handle concurrent medical issues, the system supports branching. If a patient presents a new distinct symptom, the logic creates a new "Head Node," effectively starting a parallel branch in the history graph. This allows clinicians to track multiple episodes of care simultaneously without visual clutter.

3.3 Performance Optimization

To ensure low-latency responses for the frontend visualizations, the backend implements a specialized Two-Tier Caching Strategy using Redis.

3.3.1 Tier 1: Standard Resource Cache

The first tier utilizes a generic `cacheHelper` middleware. It intercepts HTTP requests for standard FHIR resources (e.g., `/api/patients`). If a valid cache entry exists, it is returned immediately, bypassing the FHIR server and database entirely. These entries typically have a Time-To-Live (TTL) of 24 hours.

3.3.2 Tier 2: Specialized Graph Cache

The second tier, managed by `toonNodesCacheHelper`, is designed for the high-computation requirements of the history graph. Constructing the graph involves complex joins and relationship mapping. Therefore, the pre-calculated graph structure (nodes and edges formatted for the visualization library) is cached separately. This cache uses an aggressive invalidation strategy: any write operation to a patient's history immediately invalidates their specific graph cache to ensure data consistency.

3.4 Frontend

3.5 Mobile

Chapter 4

Security Framework

Chapter 5

AI Integration

Chapter 6

Testing and Verification

Chapter 7

Conclusion and Future Work

Conclusion and future work go here.

Appendix

Appendix A

Lists

List of Figures

1.1	Evolution of HL7 Standards from Messaging to RESTful Resources. . . .	4
1.2	Visual Comparison of Data structure: HL7 v2 Message vs. FHIR JSON Resource.	6
3.1	High-Level Container Architecture showing the interaction between the Nginx proxy, application services, and data persistence layers.	9
3.2	Internal layered architecture of the Node.js Application Server, detailing the middleware stack and service distribution.	10
3.3	Entity-Relationship Diagram (ERD) illustrating the integration of custom graph tables with the standard FHIR resource table.	12

References

- [1] Grand View Research, *Digital Health Market Size, Share & Trends Analysis Report By Technology, By Component, By Region, And Segment Forecasts, 2025 - 2030*, <https://www.grandviewresearch.com/industry-analysis/digital-health-market>, Accessed: 2025-04-04.
- [2] Intellisoft, *FHIR vs. HL7: Core Differences & Standards Comparison*, <https://intellisoft.io/comparing-hl7-vs-fhir-core-differences/>, Accessed: 2025-04-04.
- [3] University of Porto Repository, *Interoperability in Healthcare Systems (Thesis/Dissertation)*, <https://repositorio-aberto.up.pt/bitstream/10216/164181/2/700384.pdf>, Accessed: 2025-04-04.
- [4] Wikipedia, *Fast Healthcare Interoperability Resources*, https://en.wikipedia.org/wiki/Fast_Healthcare_Interoperability_Resources, Accessed: 2025-04-04.
- [5] HL7 International, *FHIR HL7 Documentation*, <https://www.hl7.org/fhir/>, Accessed: 2025-04-04.